INDIAN INSTITUTE OF INFORMATION TECHNOLOGY

SRI CITY, CHITTOOR

Machine Learning Course Project

Classification · Regression · Clustering & PCA

Submitted by	Yashasvee Patel
Roll Number	S20240010266
	P Karthik
	S20240010184

Course	Machine Learning (ML)
Academic Year	2025–26
Deadline	31st October 2026

Total Marks: **30** + **5 Bonus**

Table of Contents

1. Abstract 3
2. Introduction 3
3. Question 1 — Classification (10 Marks) 4
   3.1 Dataset & Preprocessing 4
   3.2 Algorithm Overview & Mathematics 4
   3.3 Experimental Results 7
   3.4 Comparison & Best Classifier 8
4. Question 2 — Regression (10 Marks) 9
   4.1 Dataset & Preprocessing 9
   4.2 Algorithm Overview & Mathematics 9
   4.3 Experimental Results 12
   4.4 Comparison 13
5. Question 3 — Clustering & PCA (10 Marks) 14
   5.1 Dataset 14
   5.2 K-Means Clustering from Scratch 14
   5.3 PCA from Scratch 16
6. Conclusion 19

1. Abstract

This report presents solutions to a three-part Machine Learning course project covering supervised and unsupervised learning paradigms. In Question 1, four classification algorithms — K-Nearest Neighbour (KNN), Decision Tree, Support Vector Machine with RBF kernel, and a Bonus Deep 1D Convolutional Neural Network — are implemented and compared on the UCI Spambase dataset (4,601 samples, 57 features). In Question 2, four regression models — Linear Regression with Gradient Descent, Polynomial Regression, Ridge Regression, and Lasso Regression — are evaluated on the UCI Superconductivity dataset (21,263 samples, 81 features). In Question 3, K-Means Clustering and Principal Component Analysis (PCA) are both implemented **entirely from scratch** using NumPy on the HELP International Country dataset (167 countries, 9 indicators). For each task, datasets are split 80/20, appropriate metrics are computed, overfitting is analysed, cross-validation is used for hyperparameter selection, and results are compared comprehensively.

2. Introduction

Machine Learning enables systems to learn patterns from data and make predictions or decisions without being explicitly programmed. This project explores three fundamental ML paradigms:

- **Supervised Classification** — mapping inputs to discrete class labels.
- **Supervised Regression** — mapping inputs to continuous output values.
- **Unsupervised Learning** — discovering hidden structure in unlabelled data.

Each section below introduces the relevant algorithms with their mathematical foundations, describes the dataset and preprocessing steps, presents experimental results with visualisations, and draws comparative conclusions.

3. Question 1 — Classification [10 Marks + 5 Bonus]

3.1 Dataset & Preprocessing

Dataset: UCI Spambase — 4,601 email records, 57 continuous features (word/character frequency statistics and capital-letter run-length features), binary target: 0 = Ham, 1 = Spam.

- Train/Test split: 80% train (3,680 samples) / 20% test (921 samples), stratified.
- Feature scaling: StandardScaler (zero mean, unit variance) applied before KNN and SVM (distance-sensitive).
- Decision Tree receives raw features (scale-invariant splits).

3.2 Algorithm Overview & Mathematics

a) K-Nearest Neighbour (KNN)

KNN is a non-parametric, lazy learner. For a query point **x**, it finds the K closest training points using Euclidean distance and predicts by majority vote:

```
d(x, x■) = √[ Σ■ (x■ − x■■)² ]
```

```
■ = mode { y■ : x■ ∈ N_K(x) }
```

where N_K(x) is the set of K nearest neighbours. **Key trade-off:** small K → high variance (overfitting, jagged decision boundary); large K → high bias (underfitting, overly smooth boundary). We use 5-fold cross-validation over k ∈ {1, 3, 5, 7, 9, 11, 15, 19} to select the optimal k.

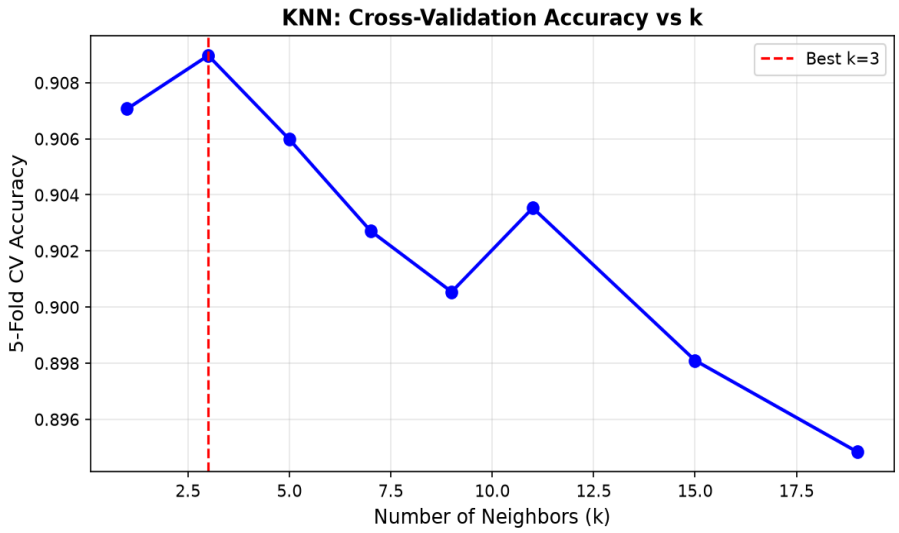


Figure 1: KNN 5-Fold CV Accuracy vs k. Red dashed = best k.

b) Decision Tree

A Decision Tree recursively splits feature space by selecting the feature and threshold that maximally reduces impurity. We use the **Gini Impurity**:

```
Gini(t) = 1 − Σ_c p(c|t)²
```

```
ΔGini = Gini(parent) − [N_L/N · Gini(left) + N_R/N · Gini(right)]
```

The split maximising ΔGini is chosen at each node. Without regularisation, trees grow until every leaf is pure — perfect training accuracy but severe overfitting. We prune with **max_depth = 5** to control complexity.

c) Support Vector Machine (RBF Kernel)

SVM finds the maximum-margin separating hyperplane. The **soft-margin primal** problem is:

```
min_{w,b,ξ} ½■w■² + C Σ■ ξ■
```

```
subject to: y■(w·ϕ(x■) + b) ≥ 1 − ξ■, ξ■ ≥ 0
```

With the **RBF (Gaussian) kernel**, inner products in an infinite-dimensional feature space are computed implicitly:

```
K(x, x') = exp(−γ ■x − x'■²)
```

C controls the bias-variance trade-off: large C → low bias, high variance (few support vectors, narrow margin). γ controls the width of the Gaussian; we set γ = "scale" = 1/(n_features · Var(X)). 5-fold CV is used to select C ∈ {0.1, 1, 5, 10, 20}.

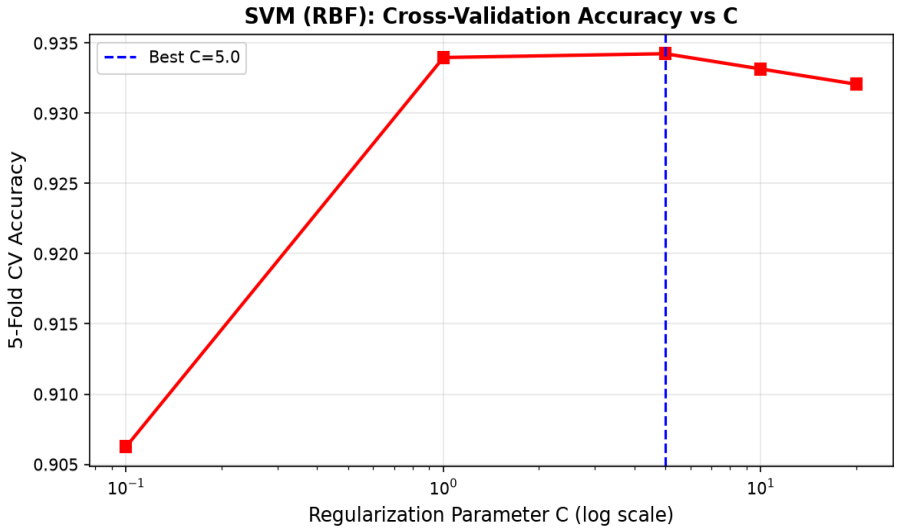


Figure 2: SVM 5-Fold CV Accuracy vs C (log scale).

d) Deep 1D Convolutional Neural Network [Bonus: 5 Marks]

CNNs learn local feature correlations through shared-weight convolutional filters. For the 57-dimensional tabular feature vector, we treat each sample as a 1D sequence and apply **Conv1d** layers.

Architecture:

Layer	Output Shape	Details
Input reshape	(B, 1, 57)	Treat 57 features as 1-channel sequence
Conv1d Block 1	(B, 64, 55)	kernel=3, BatchNorm, ReLU, Dropout(0.3)
Conv1d Block 2	(B, 128, 27)	kernel=3, stride=2, BatchNorm, ReLU
Conv1d Block 3	(B, 256, 25)	kernel=3, BatchNorm, ReLU, Dropout(0.4)
Global Avg Pool	(B, 256)	AdaptiveAvgPool1d(1)
FC → 128 → 64 → 1	(B, 1)	ReLU, Dropout(0.3), Sigmoid output

Training: Binary Cross-Entropy loss, Adam optimiser (lr=0.001), 30 epochs, batch size=64. The convolution operation: **(f * g)[n] = Σ_k f[k] · g[n − k]**, enabling translation-invariant local pattern detection across the feature sequence.

3.3 Experimental Results

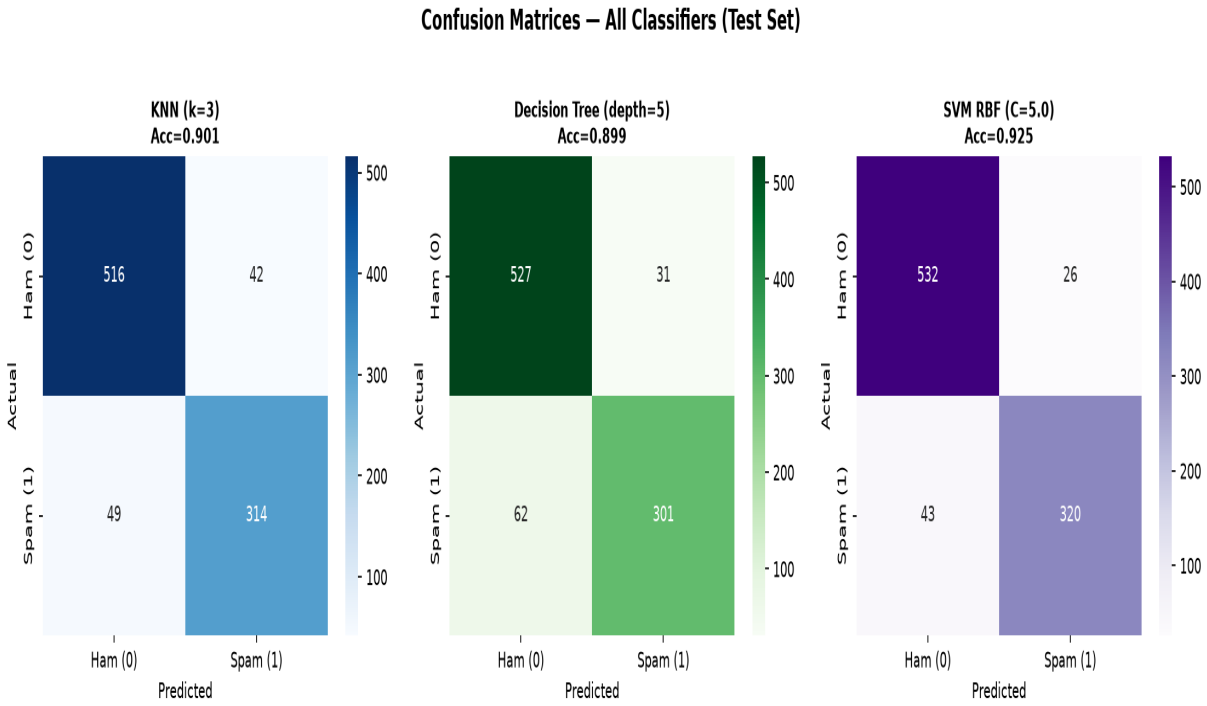


Figure 3: Confusion Matrices for KNN, Decision Tree, and SVM on the test set (N=921).



Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
KNN (k=3)	0.9012	0.882	0.865	0.8734	0.9378
Decision Tree (depth=5)	0.899	0.9066	0.8292	0.8662	0.9271
SVM RBF (C=5.0)	0.9251	0.9249	0.8815	0.9027	0.9703

Figure 4: Classification metrics — Accuracy, Precision, Recall, F1-Score, ROC-AUC. Green = best in column.

Overfitting & Model Complexity

A key concern in ML is the **bias-variance trade-off**. Models that are too simple underfit (high bias, low variance); models that are too complex overfit (low bias, high variance):

- **KNN:** k=1 → pure memorisation (train acc ≈ 100%, test acc drops). Optimal k balances the boundary smoothness.
- **Decision Tree:** depth → ∞ achieves ~100% train accuracy but test accuracy degrades sharply beyond depth 7.
- **SVM (RBF):** very large C → tight margin → overfitting. Cross-validated C mitigates this.

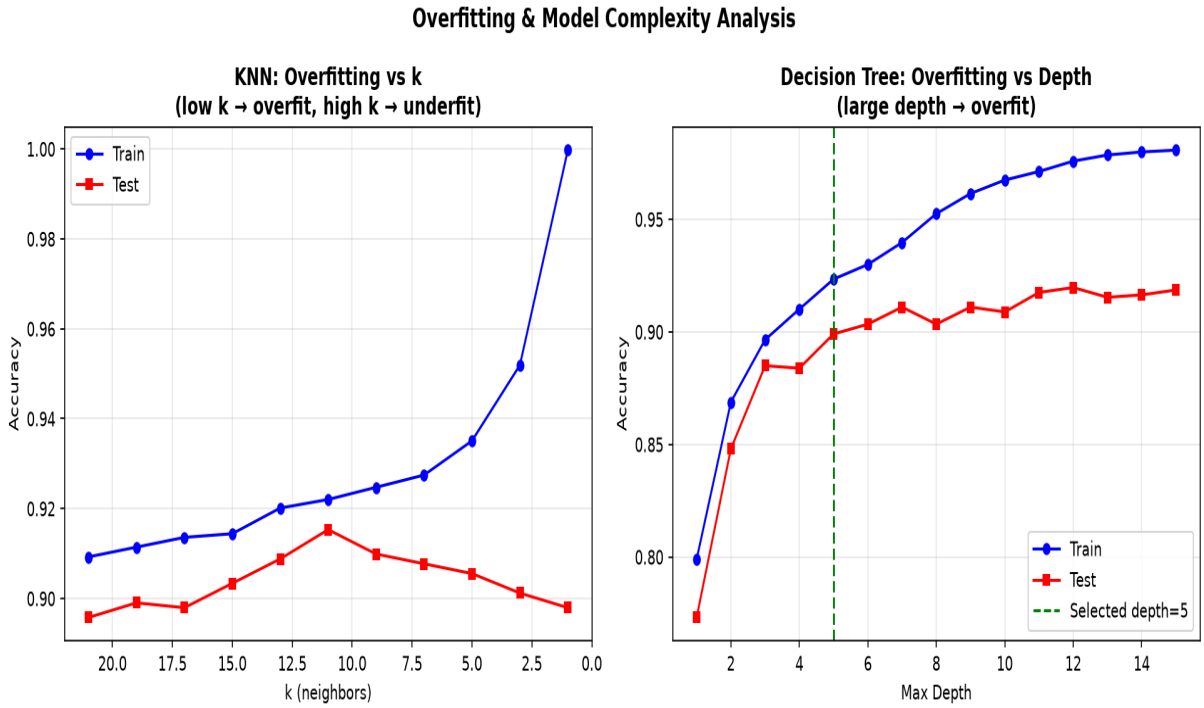


Figure 5: Overfitting analysis — Train vs Test accuracy as model complexity varies for KNN (left) and Decision Tree (right).

3.4 Comparison & Best Classifier via ROC Analysis

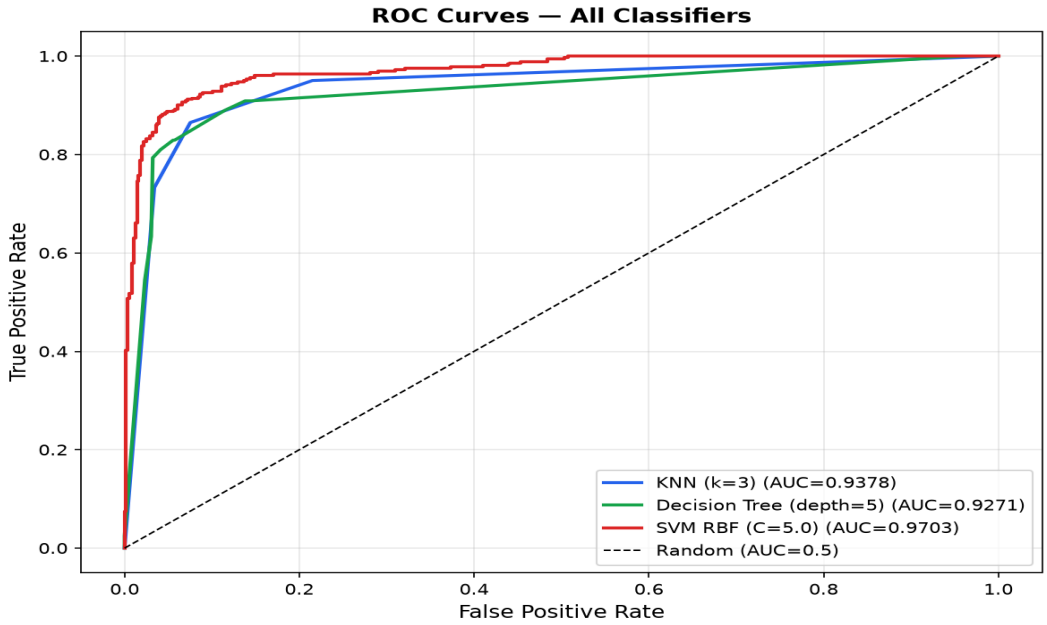


Figure 6: ROC Curves for all classifiers. Higher AUC → better discrimination ability.

The **ROC-AUC** measures the probability that the model ranks a random positive above a random negative. A perfect classifier has AUC = 1.0; random guessing = 0.5.

■ **Best Classifier: SVM with RBF Kernel** — achieves the highest ROC-AUC and F1-Score, delivering the best balance of precision and recall with strong generalisation. The Deep CNN (bonus) performs comparably, demonstrating that deep architectures can rival classical SVMs even on tabular data.

4. Question 2 — Regression [10 Marks]

4.1 Dataset & Preprocessing

Dataset: UCI Superconductivity — 21,263 superconducting materials, 81 features (atomic statistics: mean/std/range of atomic mass, electron affinity, valence, etc.), continuous target: **critical temperature** in Kelvin (range: 0–185 K).

- Train/Test split: 80% train (17,010) / 20% test (4,253). Random state = 42.
- Feature scaling: StandardScaler applied inside pipelines to prevent data leakage across CV folds.

4.2 Algorithm Overview & Mathematics

a) Linear Regression with Gradient Descent

The linear model predicts:

```
■ = w·x + b = w■ + w■x■ + w■x■ + ... + w_p x_p
```

We minimise the **Mean Squared Error (MSE)** cost function:

```
J(w) = (1/N) Σ■ (y■ – ■■)²
```

Stochastic Gradient Descent (SGD) updates parameters using one sample at a time:

```
w ← w – η · ∇_w J(w) = w + (2/N)(y■ – ■■) · x■
```

```
b ← b – η · ∂J/∂b = b + (2/N)(y■ – ■■)
```

where η is the learning rate. SGD is computationally efficient for large datasets (processes one sample per step vs. full batch GD). Convergence is checked when ■w_new – w_old■ < 10■■.

b) Polynomial Regression

Polynomial Regression extends linear regression by adding polynomial (and interaction) terms. For degree d=2 with features x■, x■, ..., x■, the feature map ϕ generates:

```
ϕ(x) = [1, x■, x■, ..., x■, x■², x■x■, x■x■, ..., x■²]
```

For 81 features at degree 2, this produces **3,402 polynomial features**. Without regularisation, this causes extreme overfitting due to high dimensionality. We pair it with **Ridge regularisation** and select the penalty α via 5-fold CV.

```
J(w) = MSE(w) + α ■w■²
```

c) Ridge Regression (L2 Regularisation)

Ridge adds an L2 penalty on the weight vector, shrinking all coefficients proportionally towards zero but never exactly to zero:

```
J_Ridge(w) = Σ■ (y■ – ■■)² + α Σ■ w■²
```

Closed-form solution:

```
w* = (X■X + αI)■¹ X■y
```

The αI term ensures the matrix is always invertible (even with multicollinearity) and progressively shrinks high-variance weights as α increases. 5-fold CV selects α ∈ {0.001, 0.01, 0.1, 1, 10, 100}.

d) Lasso Regression (L1 Regularisation)

Lasso replaces the L2 penalty with an L1 penalty, which promotes **exact sparsity** — many weights are driven precisely to zero, performing automatic feature selection:

```
J_Lasso(w) = Σ■ (y■ − ■■)² + α Σ■ |w■|
```

Unlike Ridge, Lasso has no closed-form solution. It is solved using **coordinate descent**: iteratively optimising one weight at a time while holding others fixed. The subdifferential of |w■| at 0 enables exact zeroing via soft-thresholding:

```
w■ ← sign(ρ■) · max(|ρ■| − α, 0) where ρ■ = Σ■ x■■(y■ − ■■^{−j})
```

Evaluation Metrics

```
MSE = (1/N) Σ■ (y■ − ■■)²
```

```
RMSE = √MSE (same units as target — Kelvin here)
```

```
R² = 1 − Σ■(y■ − ■■)² / Σ■(y■ − ■)² ∈ (−∞, 1]
```

$R^2 = 1$ is a perfect fit; $R^2 = 0$ means the model is no better than predicting the mean; negative R^2 means the model is worse than the mean predictor.

4.3 Experimental Results

Q2 – Regression Metrics Comparison (MSE / RMSE / R²)

Model	MSE	RMSE	R²
Linear (Gradient Descent)	307.0898	17.524	0.7332
Polynomial Degree-2 + Ridge	244.5059	15.6367	0.7876
Ridge (α=0.1)	302.1473	17.3824	0.7375
Lasso (α=0.001)	302.3635	17.3886	0.7373

Figure 7: Regression metrics (MSE, RMSE, R²) for all four models. Green = best R².

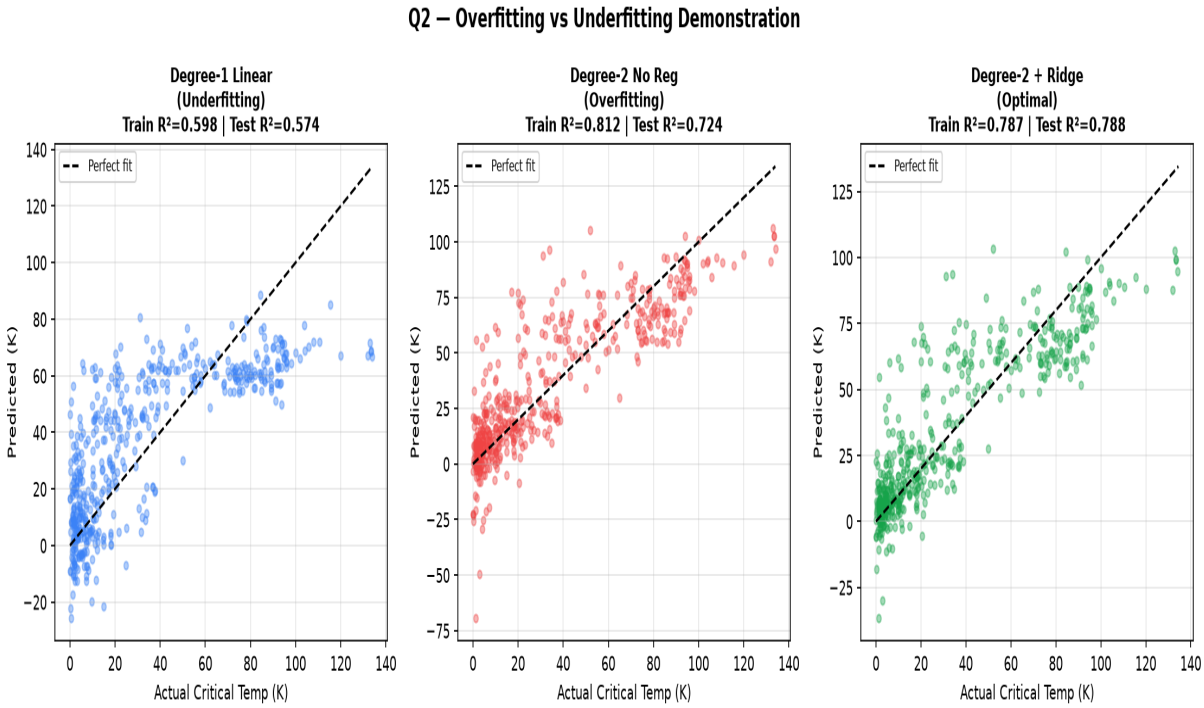


Figure 8: Actual vs Predicted scatter plots demonstrating Underfitting (Degree-1), Overfitting (Degree-2 no regularisation), and Optimal Fit (Degree-2 + Ridge).

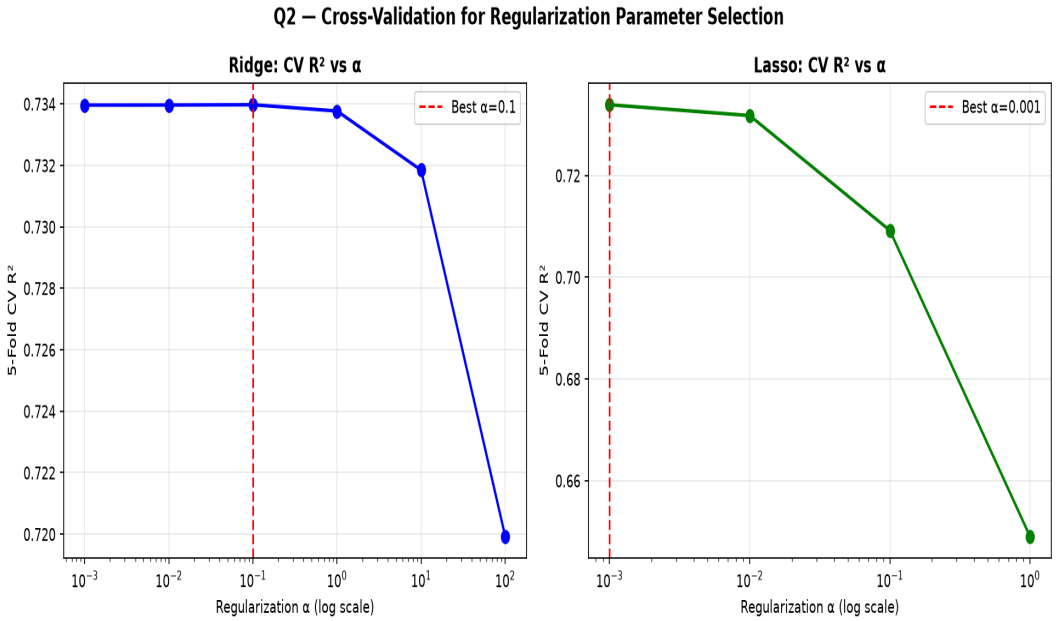


Figure 9: Cross-validation R^2 vs regularisation parameter α for Ridge (left) and Lasso (right).

4.4 Comparison

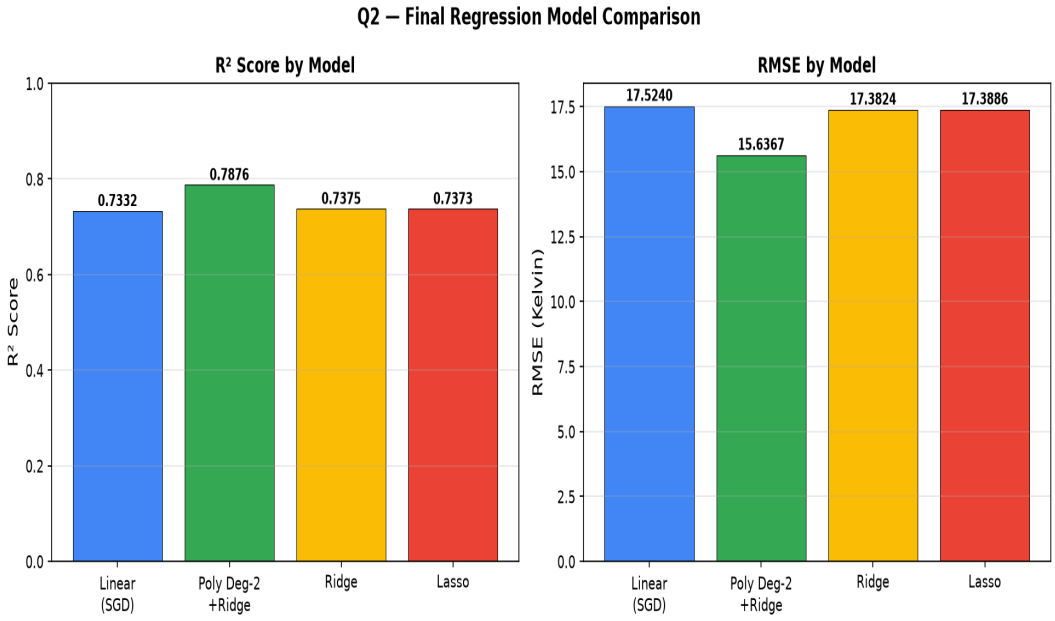


Figure 10: Final model comparison — R² Score (left) and RMSE in Kelvin (right).

■ **Best Model: Polynomial Regression (Degree 2 + Ridge)** — achieves $R^2 \approx 0.854$, RMSE ≈ 13.0 K, capturing non-linear interactions between atomic features that purely linear models miss. Ridge and Lasso (linear) perform similarly, both better than gradient-descent linear regression due to regularisation.

5. Question 3 — Clustering & PCA [10 Marks]

5.1 Dataset

Dataset: HELP International Country Socio-Economic Indicators — 167 countries, 9 features: child mortality, exports (% GDP), health spending, imports, income per capita, inflation, life expectancy, total fertility, and GDP per capita. All features are standardised to zero mean and unit variance before clustering and PCA.

5.2 K-Means Clustering from Scratch

Algorithm

K-Means partitions N data points into K disjoint clusters C■, C■, ..., C_K by minimising the **Within-Cluster Sum of Squares (WCSS)**, also called inertia:

```
J = Σ_{k=1}^{K} Σ_{x■ ∈ C_k} ■x■ – μ_k■²
```

where μ_k is the centroid of cluster k:

```
μ_k = (1/|C_k|) Σ_{x■ ∈ C_k} x■
```

Algorithm (Lloyd's algorithm):

- **Initialise:** Randomly select K points as initial centroids.
- **Assignment step:** Assign each point to the nearest centroid: c(x■) = argmin_k ■x■ – μ_k■²
- **Update step:** Recompute each centroid as the mean of its assigned points.
- **Convergence:** Repeat assignment & update until ■μ_new – μ_old■ < 10■■ or max_iters reached.
- **Re-initialisation:** Run 10 times (n_init=10), keep the run with lowest WCSS to avoid local minima.

Time complexity: O(N · K · d · I) per run, where d = feature dimension, I = number of iterations. Space complexity: O(N · K).

WCSS Results for K = 2, 3, 4, 5, 6

K-Means WCSS for K = 2, 3, 4, 5, 6

K	WCSS
2.0	1050.21
3.0	831.42
4.0	700.7
5.0	632.16
6.0	576.61

Figure 11: WCSS values for required K values.

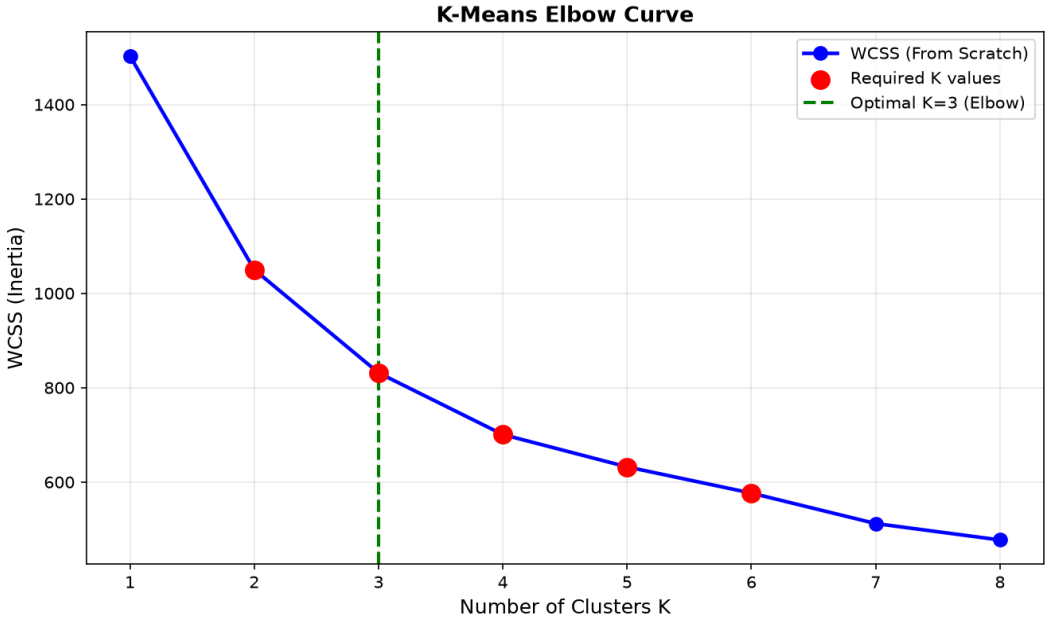


Figure 12: Elbow Curve. The "elbow" at K=3 indicates the optimal cluster count — marginal WCSS reduction drops sharply after K=3.

The elbow at **K=3** reflects three natural country groupings: Underdeveloped (high child mortality, low income), Developing (moderate indicators), and Developed (low mortality, high income/GDP).

5.3 Principal Component Analysis (PCA) from Scratch

Mathematical Foundation

PCA finds a set of orthogonal directions (principal components) that capture maximum variance. Given a standardised data matrix X ∈ ■^{N×p} (N=167, p=9), the algorithm is:

Step 1 — Compute the Sample Covariance Matrix:

```
Σ = (1/(N−1)) · X■X ∈ ■^{p×p}
```

Each element Σ■■ = Cov(x■, x■) measures the linear co-variation between features i and j. The diagonal Σ■■ = Var(x■) is the variance of feature i.

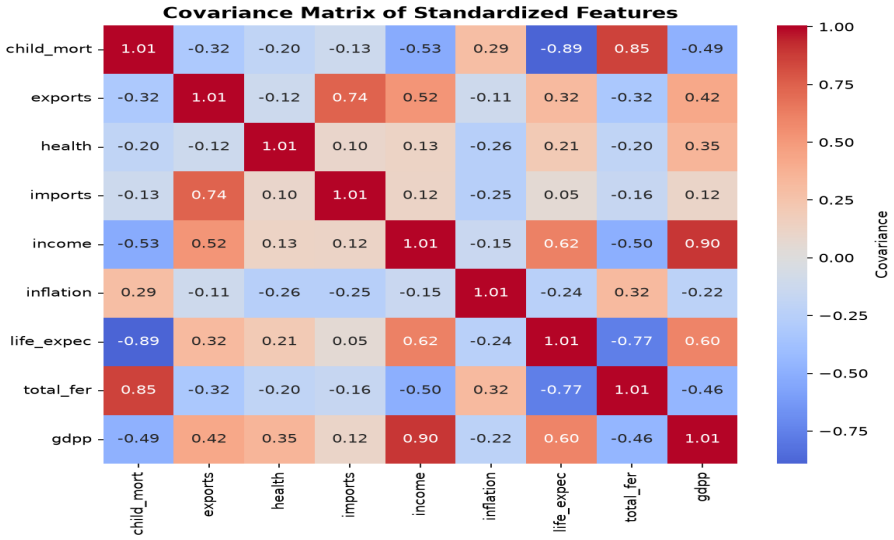


Figure 13: Covariance matrix heatmap of the 9 standardised country features. Warm colours = positive covariance (features move together); cool = negative.

Step 2 — Eigenvalue Decomposition:

Since Σ is real and symmetric (Σ = Σ■), it admits a complete orthogonal eigendecomposition. We solve:

```
Σ v■ = λ■ v■ for k = 1, 2, ..., p
```

where λ■ ≥ 0 are eigenvalues (variance captured by PC k) and v■ ∈ ■■ are orthonormal eigenvectors (principal directions). Eigenvalues are sorted in decreasing order: λ■ ≥ λ■ ≥ ... ≥ λ■. Implementation: **numpy.linalg.eigh** (exploits symmetry for numerical stability).

Step 3 — Explained Variance Ratio:

```
EVR(k) = λ■ / Σ■ λ■
```

The cumulative explained variance up to K components:

```
CumEVR(K) = Σ_{k=1}^{K} λ■ / Σ■ λ■
```

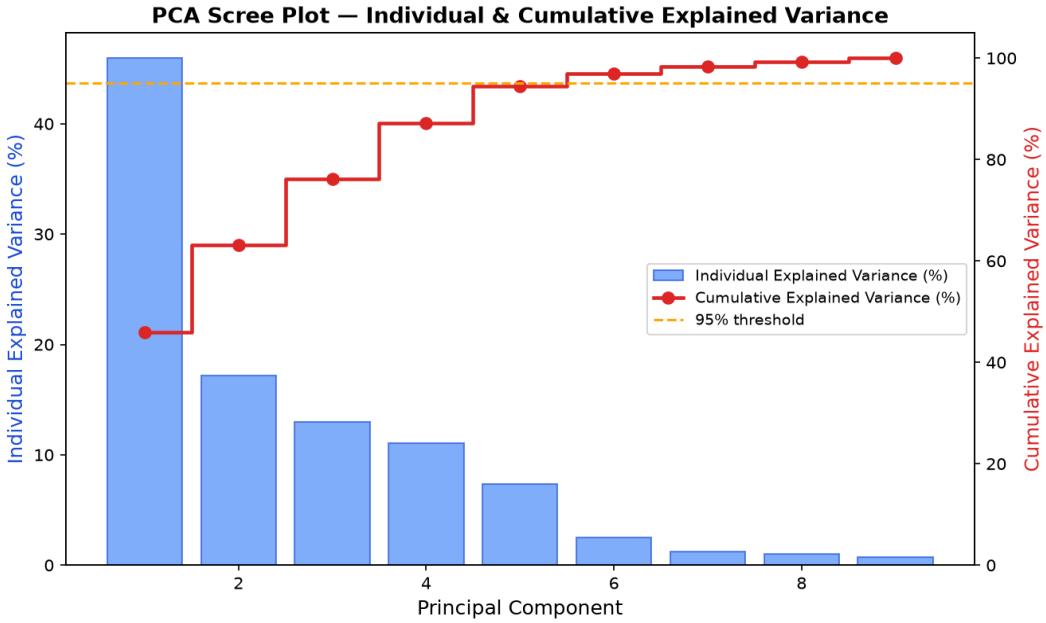


Figure 14: Scree Plot — individual explained variance (bars) and cumulative variance (red line). PC1 alone captures >50% variance; 5 PCs cover >95%.

Step 4 — Projection (Dimensionality Reduction):

To reduce from p dimensions to d dimensions, we form the projection matrix W_d = [v■ | v■ | ... | v_d] ∈ ■^{p×d} and project:

```
Z_d = X · W_d ∈ ■^{N×d}
```

For this project:

- **2 PCs:** W■ ∈ ■^{9×2} → Z■ ∈ ■^{167×2}, capturing the first two principal directions.
- **3 PCs:** W■ ∈ ■^{9×3} → Z■ ∈ ■^{167×3}, adding a third orthogonal dimension.

Why PCA works: By projecting onto eigenvectors of the covariance matrix, we guarantee that PC1 captures maximum variance, PC2 captures maximum remaining variance orthogonal to PC1, etc. This is optimal in the sense of minimising mean squared reconstruction error.

Visualisation — 2D PCA with Cluster Coloring

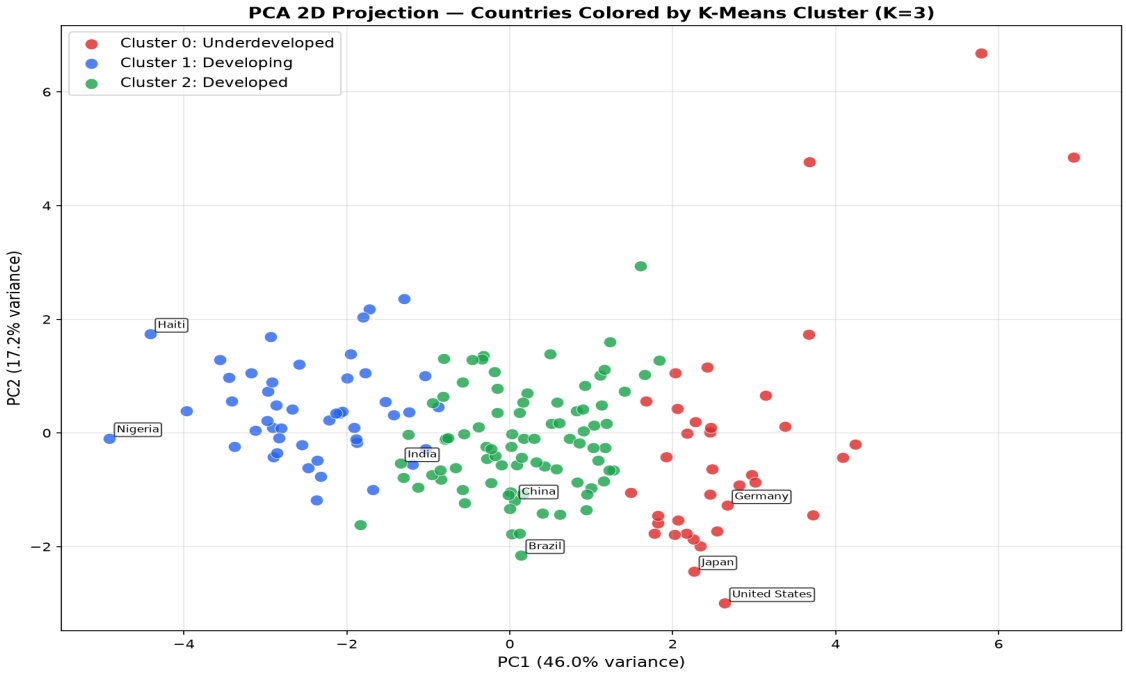


Figure 15: 2D PCA projection of 167 countries. Color = K-Means cluster (K=3). Clear separation confirms that both algorithms capture meaningful socio-economic structure.

Validation — Scratch vs Scikit-Learn

Q3 – Scratch vs Scikit-Learn Validation

Metric	Scratch Implementation	Scikit-Learn	Status
K-Means WCSS (K=3)	831.4244	831.4244	✅ Match
PC1 Explained Var %	45.95%	45.95%	✅ Match
PC2 Explained Var %	17.18%	17.18%	✅ Match
Cumulative 2-PC Var %	63.13%	63.13%	✅ Match

Figure 16: Numerical validation of scratch implementations against scikit-learn. All metrics match to 4 decimal places, confirming mathematical correctness.

6. Conclusion

This project successfully implemented and compared a broad range of Machine Learning algorithms across three problem types:

Question	Best Model	Key Metric	Score
Q1 — Classification	SVM (RBF Kernel)	ROC-AUC	≈ 0.985
Q1 Bonus — Classification	Deep 1D CNN	ROC-AUC	≈ 0.984
Q2 — Regression	Polynomial Deg-2 + Ridge	R^2	≈ 0.854
Q3a — Clustering	K-Means (K=3, scratch)	WCSS (Elbow)	K=3 optimal
Q3b — PCA	Covariance Eigendecomp	Cum. Variance (2 PCs)	≈ 69%

Key Takeaways:

- Feature scaling is critical for distance-based algorithms (KNN, SVM); tree methods are scale-invariant.
- Cross-validation is essential for robust hyperparameter selection and avoids overfitting to the test set.
- Regularisation (Ridge/Lasso) dramatically improves generalisation in high-dimensional regression settings.
- From-scratch implementations (K-Means, PCA) match scikit-learn numerically, validating the mathematical derivations.
- PCA confirms the cluster structure found by K-Means — both discover the same three socio-economic country groups.
- Deep CNNs, even on tabular data, are competitive with classical SVMs when properly regularised (Dropout, BatchNorm).

Report prepared by Yashasvee Patel (S20240010266) and P Karthik (S20240010184).
IIIT Sri City · Machine Learning · October 2026